校园二手交易系统 — 源码位置与团队分工说明

用途：老师查看云部署后要求看代码时，按本文档直接定位；同时说明 4 名成员各自负责的部分。

版本：`main` 分支（2026-10-27），交付版本 `v1.0`（tag `9143a3c`，核心后端提交 `e0fc377`）。

说明：代码行号以当前 `main` 为准；若后续有小改动，按方法名/文件名搜索即可。

0. 给老师的一分钟说明

这个项目是前后端分离的 JavaWeb 课程项目：前端 Vue 3，后端 Spring Boot + MyBatis，数据库 MySQL。

源码位置：

- *GitHub：`https://github.com/SkyLoveFree/campus-secondhand`（main 分支，tag `v1.0`）*
- *本机：`C:\Users\Sky\Desktop\JavaWeb`*
- *云服务器：`120.76.241.77` 上运行的是**编译后的 Jar**（`/opt/campus-secondhand/app.jar`），源代码不在服务器上，源代码以 GitHub/本机为准。*

建议先打开这 4 个位置：

1. *`backend/src/main/java/com/campus/secondhand/interceptor/AuthInterceptor.java` —— 登录拦截与白名单；*
2. *`backend/src/main/java/com/campus/secondhand/service/impl/ItemServiceImpl.java` —— 商品业务与权限判定；*
3. *`frontend/src/views/ItemList.vue` / `ItemForm.vue` / `ItemDetail.vue` —— 前端三个核心页面；*
4. *`sql/schema.sql` —— 两张表、索引、约束。*

1. 源码位置总表

内容	位置
GitHub 仓库	https://github.com/SkyLoveFree/campus-secondhand
GitHub 源码树	https://github.com/SkyLoveFree/campus-secondhand/tree/main
本地项目根目录	`C:\Users\Sky\Desktop\JavaWeb`
后端源码	`backend/src/main/java/com/campus/secondhand/`
后端配置/Mapper XML	`backend/src/main/resources/`
后端测试	`backend/src/test/java/com/campus/secondhand/`
前端源码	`frontend/src/`
前端构建配置	`frontend/vite.config.js`、`frontend/package.json`
数据库脚本	`sql/schema.sql`（建表）、`sql/data.sql`（测试数据）
设计文档	`docs/`
演示截图	`screenshots/`（14 张）
线上运行物	服务器 `/opt/campus-secondhand/app.jar`（编译产物）+ `/opt/campus-secondhand/sql/`
线上服务	systemd：`campus-secondhand.service`，端口 `8080`
线上访问	http://120.76.241.77:8080

backend/src/main/resources/static/ 是前端 npm run build 的产物复制目录（.gitignore 忽略，不入库），不是源码；真正的前端源码在 frontend/src/。单 Jar 部署时它被一起打进 Jar。

2. 目录结构与归属

```
campus-secondhand/
├─ AGENTS.md / README.md                  [A] 项目规则、环境与启动说明
├─ docs/                                  [A] 需求、架构、API、分工、流程、评审、手册
│  ├─ requirements.md / architecture.md
│  ├─ api-spec.md / database-design.md    [C] database-design.md
│  ├─ team-assignments.md / development-workflow.md
│  ├─ design-review.md / user-manual.md / presentation-and-qa.md
│  └─ code-map-and-ownership.md           本文档
├─ sql/                                   [C] 数据库脚本
│  ├─ schema.sql                          建库、user/item 两张表、索引、约束
│  └─ data.sql                            2 个测试账号 + 5 条商品
├─ backend/                               [D] Spring Boot 后端
│  ├─ pom.xml / mvnw / mvnw.cmd
│  └─ src/main/java/com/campus/secondhand/
│     ├─ CampusSecondhandApplication.java
│     ├─ common/                          Result、ErrorCode、异常处理
│     ├─ config/                          MyBatis/密码/Web 配置
│     ├─ interceptor/AuthInterceptor.java 登录拦截器
│     ├─ controller/                      AuthController、ItemController
│     ├─ service/ + service/impl/         AuthService、ItemService 及实现
│     ├─ mapper/ + resources/mapper/      UserMapper、ItemMapper 及 XML
│     ├─ entity/                          User、Item
│     └─ dto/                             请求/响应对象
├─ frontend/                              [B] Vue 3 前端
│  ├─ package.json / vite.config.js / index.html
│  └─ src/
│     ├─ main.js / App.vue / styles/main.css
│     ├─ router/index.js                  hash 路由 + 登录守卫
│     ├─ stores/user.js                   Pinia 登录态
│     ├─ utils/request.js                 Axios 实例与统一错误处理
│     ├─ api/auth.js / api/item.js        接口封装
│     ├─ components/AppHeader.vue / ItemCard.vue
│     └─ views/Login.vue / Register.vue / ItemList.vue / ItemDetail.vue / ItemForm.vue
└─ screenshots/                           [B] 14 张 UI 演示截图
```

3. 后端代码地图（D 负责）

3.1 请求处理链路

```
浏览器（Vue 页面）
  → Vite 代理 /api 或单 Jar 同域
  → AuthInterceptor：白名单放行 / 未登录返回 401
  → Controller：路由 + Bean Validation
  → Service：业务规则、权限、事务
  → Mapper（XML）：参数化 SQL
  → MySQL
  ← Result{code,message,data}；异常由 GlobalExceptionHandler 统一转换
```

3.2 后端文件职责

文件	职责
CampusSecondhandApplication.java	Spring Boot 启动类
common/Result.java	统一响应 {code,message,data}
common/ErrorCode.java	业务码与 HTTP 状态、固定 message
common/BusinessException.java	业务异常
common/GlobalExceptionHandler.java	参数错误、JSON 格式、405、404、500 等统一处理
common/Constants.java	Session key、角色、分页上限等常量

文件	职责
config/WebMvcConfig.java	注册拦截器与路径
config/PasswordConfig.java	BCryptPasswordEncoder Bean
interceptor/AuthInterceptor.java	Session 登录校验 + 接口白名单
controller/AuthController.java	注册、登录、登出、当前用户
controller/ItemController.java	列表/搜索/排序/分页、详情、发布、修改、删除
service/AuthService.java + impl/AuthServiceImpl.java	注册/登录/登出/me 业务，用户名与密码规则
service/ItemService.java + impl/ItemServiceImpl.java	商品查询与写业务、归属权限、事务
mapper/UserMapper.java + resources/mapper/UserMapper.xml	用户查询/计数/新增
mapper/ItemMapper.java + resources/mapper/ItemMapper.xml	商品查询/分页/排序/新增/修改/删除 SQL
entity/User.java、Item.java	数据库实体
dto/*	登录/注册/查询/保存请求，ItemVO/UserVO/PageResult 响应
resources/application.yml	端口、数据源、Session、Jackson、MyBatis
src/test/java/.../*Test.java	77 个后端测试

3.3 关键后端代码位置（带行号）

关注点	位置
登录白名单（4 个路径）	interceptor/AuthInterceptor.java L38-L42
拦截器 preHandle 与 Session 校验	AuthInterceptor.java L53-L68
注册接口	controller/AuthController.java L39-L44
登录接口（写入 Session）	AuthController.java L47-L61
登出 / 当前用户	AuthController.java L64-L89
注册业务（重复用户名、BCrypt）	service/impl/AuthServiceImpl.java L55-L77
登录业务（密码校验、Session 数据）	AuthServiceImpl.java L80-L96
商品列表/搜索/排序/分页	controller/ItemController.java L45-L51 → ItemServiceImpl#listItems L62-L81
商品详情	ItemController.java L53-L59 → ItemServiceImpl#getItem L84-L93
发布商品（归属取 Session）	ItemController.java L61-L68 → ItemServiceImpl#createItem L97-L107
修改商品（本人/管理员）	ItemController.java L70-L79 → ItemServiceImpl#updateItem L111-L123
删除商品（本人/管理员）	ItemController.java L82-L92 → ItemServiceImpl#deleteItem L127-L134
归属权限核心	ItemServiceImpl#loadAndCheckPermission L140-L153：先 404，再 ADMIN 放行，再本人，否则 403
查询参数二次校验	ItemServiceImpl#validateQuery L158-L176
列表 SQL（LIKE 转义）	resources/mapper/ItemMapper.xml L12-L16
排序白名单 + LIMIT	ItemMapper.xml L25-L36
新增/修改/删除 SQL	ItemMapper.xml L44-L61
用户 SQL（唯一用户名、BCrypt 字段）	resources/mapper/UserMapper.xml L11-L38
统一异常映射	common/GlobalExceptionHandler.java L32-L126
错误码表	common/ErrorCode.java L13-L37
会话/数据源配置	resources/application.yml L1-L32

3.4 商品查询/写入要点（老师常问）

- **搜索**：keyword trim 后转义 \、%、_，SQL 用 LIKE CONCAT('%', #{keyword}, '%') ESCAPE '\\'，用 #{} 防注入。

- **排序**：只接受 `price_asc` / `price_desc`，在 XML 的 `<choose>` 里映射成固定 SQL；其他值在 Service 校验为 400。
- **分页**：`page` 默认 1、`size` 默认 10、最大 50；SQL 用 `LIMIT #{offset}, #{size}`。
- **权限**：修改/删除先查商品（不存在 404），再判断 `ADMIN` 或 `item.user_id == Session 当前用户 ID`，否则 403；请求里的 `userId` 完全忽略。
- **事务**：写方法带 `@Transactional`；UPDATE/DELETE 影响行数为 0 时归一为 404。

4. 前端代码地图（B 负责）

4.1 文件职责

文件	职责
`frontend/src/main.js`	创建 Vue 应用，挂载 Pinia、Router、Element Plus
`frontend/src/App.vue`	根组件：Header + RouterView + Footer
`frontend/src/router/index.js`	hash 路由、页面懒加载、登录守卫
`frontend/src/stores/user.js`	Pinia 登录态：`setUser/clear/restore`、角色判断
`frontend/src/utils/request.js`	Axios `baseURL=/api`、`withCredentials`、响应拦截、401/403/400/500 统一处理
`frontend/src/api/auth.js`	注册/登录/登出/当前用户接口封装
`frontend/src/api/item.js`	列表/详情/发布/修改/删除接口封装
`frontend/src/components/AppHeader.vue`	顶部导航、登录/发布/退出、角色标签
`frontend/src/components/ItemCard.vue`	商品卡片（名称/价格/时间）
`frontend/src/views/ItemList.vue`	列表、搜索、排序、分页、URL query 同步、三态
`frontend/src/views/ItemDetail.vue`	详情、联系方式、404 空态、本人/管理员编辑删除入口
`frontend/src/views/ItemForm.vue`	发布/编辑共用表单、编辑回填、字段错误回填
`frontend/src/views/Login.vue` / `Register.vue`	登录/注册，页面校验与错误提示
`frontend/src/styles/main.css`	全局样式
`frontend/vite.config.js`	`@` 别名、开发代理 `/api → localhost:8080`

4.2 关键前端代码位置（带行号）

关注点	位置
hash 路由与登录守卫	`router/index.js` **L42-L60**
登录态恢复（/api/auth/me）	`stores/user.js`（restore）+ `router/index.js` L50-L53
Axios 实例与 Cookie	`utils/request.js` **L7-L11**
统一响应拦截 / 401 / 403 / 400	`utils/request.js` **L79-L103**
列表 URL query 作为状态源	`views/ItemList.vue` **L21-L32**
列表请求与竞态保护	`ItemList.vue` **L45-L75**
搜索/排序/分页更新 URL	`ItemList.vue` **L80-L109**
详情 404 与权限入口	`views/ItemDetail.vue` **L21-L27**、**L81-L105**、**L160-L165**
编辑回填	`views/ItemForm.vue` **L67-L82**
发布/编辑提交与错误处理	`ItemForm.vue` **L96-L131**
400 fieldErrors 回填	`ItemForm.vue` **L85-L94**
Header 登录/发布/退出	`components/AppHeader.vue` **L14-L31**、**L44-L56**

4.3 前端约定

- 所有请求经 `utils/request.js`，页面不拼后端 URL；
- 列表状态以 URL query 为单一数据源，刷新/前进后退可恢复；
- 列表卡片只展示名称、价格、发布时间，联系方式只在详情展示；

- 前端按钮显隐只影响体验，真正权限由后端强制校验。

5. 数据库代码位置（C 负责）

内容	位置
建库建表（user、item、索引、CHECK、外键）	sql/schema.sql L9–L54
测试账号（admin/student，口令 123456）与 5 条商品	sql/data.sql
表结构/字段映射/索引/EXPLAIN/最终校验记录	docs/database-design.md
数据库初始化/演示数据说明	docs/database-design.md §7.1、§8.1

关键设计：

- 两张表：user（id、username 唯一、password_hash、role、时间）与 item（id、item、item_price、contact_info、user_id、时间）；
- item.user_id → user.id 外键 RESTRICT；商品物理删除；用户本期不提供删除；
- item_price DECIMAL(10,2)；user.username 唯一键；role 有 CHECK；
- 索引：uk_user_username、idx_item_user_id、idx_item_created_at、idx_item_price。

6. 测试与验收位置

内容	位置
后端测试（77 个）	backend/src/test/java/com/campus/secondhand/
认证 Controller 测试	controller/AuthControllerTest.java
商品查询 Controller 测试	controller/ItemControllerTest.java
商品写接口测试	controller/ItemWriteControllerTest.java
拦截器测试	interceptor/AuthInterceptorTest.java
Service 业务测试	service/*Test.java、service/impl/ItemServiceWriteTest.java
Mapper XML 解析测试	mapper/MapperXmlParseTest.java
最终验收记录（77 测试、E2E、UI）	docs/design-review.md §7.5
演示截图 14 张	screenshots/
使用手册	docs/user-manual.md
答辩讲解与问答	docs/presentation-and-qa.md

运行命令：

```
cd backend
.\mvnw.cmd test
```

7. 团队分工说明

项目采用 4 人分工、契约先行、功能分支 + PR 合并。下面按「负责模块 → 关键文件 → 可核对的提交/任务」说明。

A（项目负责人 – 我）

负责：需求与范围、总体架构、API 契约、数据库设计审核、任务拆分与协作流程、PR 评审、跨模块联调、最终验收、README/交付文档、云服务器部署。

主要文件：

- docs/requirements.md、docs/architecture.md、docs/api-spec.md、docs/team-assignments.md、docs/development-workflow.md、docs/design-review.md、docs/user-manual.md、docs/presentation-and-qa.md
- README.md、AGENTS.md、docs/code-map-and-ownership.md

- 本地启动/远程部署脚本（`start.ps1`、`stop.ps1` 等）

说明：

- 按约定，A **不写业务代码**，负责把需求固化成文档/契约，组织分工，评审与合并 PR，做最终联调验收和部署；
- 最终验收时发现 D 的最后一个 PR 为空改动，经 A 决策授权，由 A 完成了 3 处错误文案/注释的收尾修复（`e0fc377`），并同步测试；
- 线上服务器（`120.76.241.77:8080`）的 systemd 部署、MySQL 初始化、运维脚本由 A 完成。

一句话：A 负责「把四个人的工作变成可运行、可验收、可交付的系统」。

B（前端）

负责：Vue 3 前端全部页面、请求层、Pinia 登录态、hash 路由与守卫、组件与样式、浏览器验收、演示截图。

主要文件：

- `frontend/src/views/Login.vue`、`Register.vue`、`ItemList.vue`、`ItemDetail.vue`、`ItemForm.vue`
- `frontend/src/utils/request.js`、`api/auth.js`、`api/item.js`
- `frontend/src/router/index.js`、`stores/user.js`
- `frontend/src/components/AppHeader.vue`、`ItemCard.vue`
- `frontend/src/styles/main.css`、`vite.config.js`
- `screenshots/` 14 张截图

关键任务/提交：

- 发布/编辑表单接入与错误回填：`c1d8b01`
- 详情页删除确认与权限入口：`94c11f7`
- README 前端说明：`f7e417f`
- M5 UI 走查截图：`b79a94c`

一句话：B 负责「用户在浏览器里看到和操作的一切」。

C（数据库）

负责：数据库设计、`schema.sql`/`data.sql`、表结构/索引/约束、Mapper SQL 评审、EXPLAIN 验证、最终数据校验与演示数据说明。

主要文件：

- `sql/schema.sql`、`sql/data.sql`
- `docs/database-design.md`（含 §6.1 EXPLAIN、§6.2 Mapper 评审、§6.3 写操作验证、§7.1 最终校验、§8.1 演示数据）

关键任务/提交：

- C4 EXPLAIN 验证记录：`df07be8`
- C5 Mapper SQL 评审 + M4-C2 写操作验证：`e6f047a`
- M5 最终校验与演示数据说明：`52aaed9`

一句话：C 负责「数据存什么、怎么存、SQL 是否正确、索引是否有效」。

D（后端）

负责：Spring Boot 工程、统一响应/异常、Session 认证、商品查询接口、商品发布/修改/删除接口、归属权限校验、后端测试。

主要文件：

- `backend/src/main/java/com/campus/secondhand/` 全部后端代码
- 重点：`interceptor/AuthInterceptor.java`、`controller/AuthController.java`、`controller/ItemController.java`、`service/impl/AuthServiceImpl.java`、`service/impl/ItemServiceImpl.java`、`resources/mapper/ItemMapper.xml`
- `backend/src/test/java/com/campus/secondhand/` 77 个测试

关键任务/提交：

- 后端骨架与认证模块：`97ec21a`（早期由 A 的本机统一提交，见下方说明）
- 商品发布/修改/删除与归属权限：`faf837e`、`467cf25`
- M4 后端测试与联调：与 B 的前端联调通过

一句话：D 负责「接口怎么实现、登录怎么校验、权限怎么判、SQL 怎么执行」。

关于 Git 作者显示的说明（老师问贡献时可如实说明）

- 早期 M2/M3 的部分提交是在 A 的统一开发机上完成的，所以 GitHub 上可能显示为 A 的账号；`faf837e`/`467cf25` 以及 B/C 的 M4/M5 分支提交则能看到对应成员的提交。
- 个人贡献以**模块归属、任务分配文档、PR 合并记录和代码内容**为准，而不是只看 Git 作者名。
- 相关证据：`docs/team-assignments.md` §9（任务与负责人）、GitHub PR #1～#9、`docs/design-review.md`（验收记录）。

8. 老师现场追问时的打开顺序（建议）

老师问	打开
项目做什么、范围	`docs/requirements.md`
整体架构、为什么前后端分离	`docs/architecture.md`
接口有哪些、字段/错误码	`docs/api-spec.md`
登录/未登录怎么拦截	`AuthInterceptor.java` L38-L68
怎么防止改别人商品	`ItemServiceImpl.java` L140-L153 + `ItemWriteControllerTest.java`
搜索/排序/分页 SQL	`ItemMapper.xml` L12-L36 + `ItemServiceImpl.java` L62-L81
前端列表怎么同步 URL	`ItemList.vue` L21-L109
发布/编辑怎么提交	`ItemForm.vue` L67-L131
删除二次确认	`ItemDetail.vue` L81-L105
数据库表/索引	`sql/schema.sql` L19-L54 + `docs/database-design.md`
怎么测试的	`backend/src/test/java/...` + `docs/design-review.md` §7.5
云服务器怎么部署	`/opt/campus-secondhand/app.jar`、`systemctl status campus-secondhand`、`docs/design-review.md` §7.3

9. GitHub 直达链接（可直接发给老师）

仓库与目录

- 仓库首页：https://github.com/SkyLoveFree/campus-secondhand

- 后端源码目录：https://github.com/SkyLoveFree/campus-secondhand/tree/main/backend/src/main/java/com/campus/secondhand
- 前端源码目录：https://github.com/SkyLoveFree/campus-secondhand/tree/main/frontend/src
- SQL：https://github.com/SkyLoveFree/campus-secondhand/tree/main/sql
- 文档：https://github.com/SkyLoveFree/campus-secondhand/tree/main/docs
- 测试：https://github.com/SkyLoveFree/campus-secondhand/tree/main/backend/src/test/java/com/campus/secondhand

关键文件

- `AuthInterceptor.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/interceptor/AuthInterceptor.java
- `AuthController.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/controller/AuthController.java
- `ItemController.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/controller/ItemController.java
- `AuthServiceImpl.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/service/impl/AuthServiceImpl.java
- `ItemServiceImpl.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/service/impl/ItemServiceImpl.java
- `ItemMapper.xml`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/resources/mapper/ItemMapper.xml
- `GlobalExceptionHandler.java`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/backend/src/main/java/com/campus/secondhand/common/GlobalExceptionHandler.java
- `frontend/src/router/index.js`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/frontend/src/router/index.js
- `frontend/src/utils/request.js`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/frontend/src/utils/request.js
- `frontend/src/views/ItemList.vue`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/frontend/src/views/ItemList.vue
- `frontend/src/views/ItemDetail.vue`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/frontend/src/views/ItemDetail.vue
- `frontend/src/views/ItemForm.vue`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/frontend/src/views/ItemForm.vue
- `sql/schema.sql`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/sql/schema.sql
- `docs/api-spec.md`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/docs/api-spec.md
- `docs/database-design.md`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/docs/database-design.md
- `docs/design-review.md`：https://github.com/SkyLoveFree/campus-secondhand/blob/main/docs/design-review.md

10. 线上部署与源码的关系

- 线上服务器：`120.76.241.77`，访问地址 `http://120.76.241.77:8080`；

- 服务器上是**编译后的 Jar**：`/opt/campus-secondhand/app.jar`（内含前端静态资源），不是源码；
- 源代码在 GitHub `main` 分支和本机项目目录；
- 服务由 systemd 托管：`systemctl status campus-secondhand`，开机自启、异常自动重启；
- 数据库在服务器本机 MySQL 8.0.46，库名 `campus_secondhand`；
- 如果老师要在服务器上看源码，需要把项目源码压缩包上传解压（当前只部署了 Jar 与 SQL）。

11. 一页速查（答辩前最后看）

- **后端入口**：`CampusSecondhandApplication.java` → `AuthInterceptor` → `AuthController` / `ItemController`
- **权限核心**：`ItemServiceImpl#loadAndCheckPermission`（404 → ADMIN → 本人 → 403）
- **搜索/排序**：`ItemServiceImpl#listItems` + `ItemMapper.xml`（LIKE 转义、排序白名单、LIMIT）
- **前端入口**：`frontend/src/main.js` → `router/index.js` → `views/*`
- **前端请求**：`utils/request.js`（统一 `{code,message,data}`、401/403/400）
- **数据库**：`sql/schema.sql`（两张表、索引、外键、CHECK）
- **测试**：`backend/src/test/java`（77 个）+ `docs/design-review.md` §7.5
- **线上**：`http://120.76.241.77:8080`，systemd `campus-secondhand`